Callbacks, Generators & Iterators

These three topics are the backbone of "how does JavaScript do asynchrony and lazy sequences", and interviewers use them to separate people who memorized `async/await` from people who understand the machinery underneath it. Callbacks are the oldest async primitive; the iterator protocol is the contract that makes `for ... of` and spread work; generators are pausable functions that unify both — they are lazy iterators *and* the historical engine behind async control flow. Expect to be asked to implement a custom iterable by hand and to predict exactly what a generator yields and when.

Callbacks and callback hell

A **callback** is just a function you pass to another function to be called later — the original async pattern in JS, used by `setTimeout`, event listeners, and old Node APIs (`fs.readFile(path, cb)`). By convention Node uses **error-first callbacks**: `cb(err, result)`, where a truthy `err` means failure.

The problem is that sequential async steps nest, producing the **pyramid of doom** — each dependent call lives inside the previous callback, error handling is duplicated at every level, and the code drifts rightward off the screen.

JS Copy

```
getUser(1, (err, user) => {
  if (err) return console.error(err);
  getPosts(user.id, (err, posts) => {
    if (err) return console.error(err);
    console.log(`${user.name} wrote "${posts[0].title}"`); // => Ada wrote
```

The deeper issue is **inversion of control**: when you hand your callback to `getUser`, *it* decides if, when, and how many times to call you. A buggy or malicious API could call your callback twice, never call it, call it synchronously when you expected async, or swallow the error you threw. You have given up control of your own continuation.

Promises fix both problems. A promise is a first-class value you own — it can settle only once (guaranteeing your handler runs at most one time), it flattens nesting into a `.then()` chain, and it routes every error to a single `.catch()`. `async/await` then makes that chain read like synchronous code while staying non-blocking.

JS Copy

```
async function show() {
  const user = await getUserP(1);
  const posts = await getPostsP(user.id);
  console.log(`${user.name} wrote "${posts[0].title}"`); // ⇒ Ada wrote "
}
```

The iterator protocol

`for … of`, spread (`…`), destructuring, and `Array.from` don't know about arrays specifically — they work against a **protocol**. An object is **iterable** if it has a method keyed by the well-known symbol `Symbol.iterator` that returns an **iterator**: an object with a `next()` method that returns `{ value, done }` on each call. When `done` becomes `true`, iteration stops (and that final `value` is ignored by `for … of`).

Here is a custom iterable `range` implemented by hand — no generators, just the raw protocol.

JS Copy

```
function range(start, end, step = 1) {
  return {
    [Symbol.iterator]() {
```

```
      if (current < end) {
        const value = current;
        current += step;
        return { value, done: false };
      }
      return { value: undefined, done: true };
    }
  };
 }
};
}
```

Because it satisfies the protocol, every iteration construct now accepts it for free.

JS Copy

```
console.log([...range(0, 10, 2)]);    // ⇒ [ 0, 2, 4, 6, 8 ]
console.log(Array.from(range(1, 4))); // ⇒ [ 1, 2, 3 ]

const out = [];
for (const n of range(1, 5)) out.push(n);
console.log(out);                     // ⇒ [ 1, 2, 3, 4 ]
```

You can also drive the protocol manually — this is literally what `for ... of` does under the hood.

JS Copy

```
const it = range(1, 3)[Symbol.iterator]();
console.log(it.next()); // ⇒ { value: 1, done: false }
console.log(it.next()); // ⇒ { value: 2, done: false }
console.log(it.next()); // ⇒ { value: undefined, done: true }
```

Note the split: **iterable** (has `[Symbol.iterator]`) vs **iterator** (has `next`). An array is iterable; `arr[Symbol.iterator]()` gives you a fresh iterator each time. Making `[Symbol.iterator]()` return a *new* iterator with its own state is what lets you loop the same object twice.

Generators

Writing that `range` iterator by hand is verbose and easy to get wrong. A **generator** — declared with `function*` — is a function you can pause and resume, and it produces an iterator automatically. Each `yield` hands a value out and freezes execution exactly there until the next `next()` call resumes it. The same `range` collapses to three lines:

JS Copy

```
function* range(start, end, step = 1) {
  for (let i = start; i < end; i += step) yield i;
}
console.log([...range(0, 10, 2)]); // ⇒ [ 0, 2, 4, 6, 8 ]
```

The defining property is **lazy evaluation**: nothing in a generator body runs until you pull. Calling the generator only creates the iterator; the first line executes on the first `next()`.

JS Copy

```
function* logged() {
  console.log('A');
  yield 1;
  console.log('B');
  yield 2;
}
const g = logged();
console.log('created');       // ⇒ created   (body has NOT run yet)
console.log(g.next().value);  // ⇒ A         then 1
```

Laziness makes **infinite sequences** practical — the generator only ever computes as many values as you consume.

JS Copy

```
function* fib() {
  let a = 0, b = 1;
  while (true) {
    yield a;
```

```
}
const f = fib();
const first10 = [];
for (let i = 0; i < 10; i++) first10.push(f.next().value);
console.log(first10); // ⇒ [ 0, 1, 1, 2, 3, 5, 8, 13, 21, 34 ]
```

A classic use is a stateful **id generator** — an ever-incrementing counter with no shared global.

JS Copy

```
function* idGen() {
  let id = 1;
  while (true) yield id++;
}
const ids = idGen();
console.log(ids.next().value, ids.next().value, ids.next().value); // ⇒ 1
```

Two-way communication with `.next(value)`

`next()` is not just a pull — it can push. The argument you pass to `next(value)` becomes the **result of the `yield` expression** that the generator is currently paused on. This makes a generator a two-way channel: it yields data out, and the caller feeds data back in.

JS Copy

```
function* echo() {
  const a = yield 'ready';      // 'ready' comes out; a := whatever next()
  const b = yield `got ${a}`;
  return `done ${b}`;
}
const e = echo();
console.log(e.next());   // ⇒ { value: 'ready', done: false }
console.log(e.next(10)); // ⇒ { value: 'got 10', done: false }   (a === 1
console.log(e.next(20)); // ⇒ { value: 'done 20', done: true }   (b === 2
```

The one subtlety: the **first `next()`'s argument is discarded**, because there is no suspended `yield` waiting to receive it yet — the generator hasn't paused anywhere.

Generators as iterators and `yield*` delegation

A generator's return value *is* an iterator (and is itself iterable), so a generator method is the cleanest way to make a class iterable — just define `*[Symbol.iterator]()`.

JS Copy

```
class Stack {
  #items = [];
  push(x) { this.#items.push(x); return this; }
  *[Symbol.iterator]() {
    for (let i = this.#items.length - 1; i ≥ 0; i--) yield this.#items[i]
  }
}
const s = new Stack().push(1).push(2).push(3);
console.log([...s]); // ⇒ [ 3, 2, 1 ]   (top of stack first)
```

`yield*` delegates to another iterable — it yields every value from the target as if they were written inline. It works with any iterable, not just other generators, and it evaluates to the delegate's *return* value.

JS Copy

```
function* inner() { yield 'a'; yield 'b'; }
function* outer() {
  yield 1;
  yield* inner();   // delegate to another generator
  yield* [4, 5];    // delegate to any iterable
  yield 6;
}
console.log([...outer()]); // ⇒ [ 1, 'a', 'b', 4, 5, 6 ]
```

`yield*` is the natural tool for recursive iteration (tree/graph traversal) and for composing generators without manually re-yielding each item.

generator that `yield`ed promises; the runner awaited each promise and fed the resolved value back in via `next(value)` — exactly the two-way channel shown above. `async/await` is essentially that pattern baked into the language, with `await` playing the role of `yield` and the engine playing the role of `co`'s runner.

Async iterators and `for await ... of`

A regular iterator's `next()` returns `{ value, done }` synchronously. An **async iterator** returns a *promise* of `{ value, done }`, and its method is keyed by `Symbol.asyncIterator`. You consume it with `for await ... of`, which awaits each step. The easy way to make one is an **async generator** (`async function*`), where you can both `await` and `yield` — ideal for streaming data like paginated API responses or Node streams.

JS Copy

```
async function* asyncCounter(n) {
  for (let i = 1; i ≤ n; i++) {
    await new Promise(r ⇒ setTimeout(r, 0)); // simulate async work
    yield i;
  }
}
async function main() {
  const out = [];
  for await (const x of asyncCounter(3)) out.push(x);
  console.log(out); // ⇒ [ 1, 2, 3 ]
}
main();
```

Common interview questions

Q: What's the difference between an iterable and an iterator?

An **iterable** implements `[Symbol.iterator]()`, which returns an **iterator** — an object with `next()` returning `{ value, done }`. Iterables can be looped by `for ... of`/spread; the iterator holds the position/state. Many iterators are also iterable

Q: Implement a custom iterable.
Give the object a `[Symbol.iterator]()` method returning `{ next() }`, or (cleaner) make that method a generator and just `yield` the values — see the `range` and `Stack` examples above. The interviewer is checking that you know `for ... of` dispatches on `Symbol.iterator`, not on `Array`.

Q: What does a generator yield lazily — what actually runs, and when?
Nothing runs when you *call* the generator; it only builds an iterator. Each `next()` runs the body up to and including the next `yield`, then freezes. So values are computed **on demand, one at a time** — that is what makes infinite sequences (`fib`, `idGen`) safe: you only ever materialize the values you pull.

Q: How do generators enable two-way communication?
`yield` sends a value out; the value passed to the following `next(v)` becomes the result of that `yield` expression inside the generator, so the caller injects data back in. The first `next()`'s argument is ignored because no `yield` is paused yet.

Q: How did generators relate to async/await?
Generators pause on `yield`; a runner (like `co`) could `yield` a promise, await it externally, and resume with `next(resolvedValue)`. `async/await` is that exact mechanism promoted into syntax — `await` is the pause point and the engine is the runner.

Gotchas / traps

Generators are one-shot. Once exhausted, an iterator stays done — re-consuming yields nothing. To loop twice, get a fresh iterator (call the generator again).

JS Copy

```
function* g() { yield 1; yield 2; }
const it = g();
console.log([...it]); // ⇒ [ 1, 2 ]
console.log([...it]); // ⇒ []   (already consumed)
```

iteration. Code after an executed `return` (or after the loop ends) is unreachable.

JS Copy

```
function* r() { yield 1; return 99; yield 2; }
console.log([...r()]); // ⇒ [ 1 ]   (99 excluded; `yield 2` unreachable)
const ri = r();
console.log(ri.next()); // { value: 1, done: false }
console.log(ri.next()); // { value: 99, done: true }
```

`yield*` captures that dropped return value. What `for ... of` throws away, `yield* someGen()` evaluates to — a common trick question.

JS Copy

```
function* withReturn() { yield 1; return 'R'; }
function* consumer() {
  const r = yield* withReturn();
  console.log('delegate returned', r); // ⇒ delegate returned R
  yield 2;
}
console.log([...consumer()]); // ⇒ [ 1, 2 ]
```

The first `next()`'s argument vanishes. People expect the first `next(x)` to feed the first `yield`, but the generator hasn't paused on a `yield` yet, so `x` is silently thrown away. Only from the second `next()` onward does the argument reach a `yield`.

`return()` and `throw()` terminate early — and `finally` still runs. Calling `it.return(v)` forces the generator to complete, running any `finally` block first (good for cleanup like closing a file/connection).

JS Copy

```
function* clean() {
  try { yield 1; yield 2; }
  finally { console.log('cleanup'); }
}
```

Callback traps that still bite: don't call your callback both synchronously and asynchronously (pick one — inconsistent timing causes race bugs), always guard the error-first `err` before using the result, and remember a hand-written callback API gives no once-only guarantee — a promise does.

QUICK CHECK — no marks, just for you.

1. What does this generator two-way channel log?

JAVASCRIPT Copy

```
function* echo() {
  const a = yield 'ready';
  const b = yield `got ${a}`;
  return `done ${b}`;
}
const e = echo();
console.log(e.next().value);
console.log(e.next(10).value);
console.log(e.next(20).value);
```

- ready
got undefined
done undefined
- ready
got 10
done 10
- ready
got ready
done got 10

Check answer

2. What does spreading this delegating generator log?

JAVASCRIPT Copy

```
function* inner() { yield 'a'; yield 'b'; }
function* outer() {
  yield 1;
  yield* inner();
  yield* [4, 5];
  yield 6;
}
console.log([...outer()].join(','));
```

- ○ 1,a,b,4,5,6
- ○ 1,4,5,6
- ○ a,b,1,4,5,6
- ○ 1,a,b,4,5,6,undefined

Check answer

3. Driving a hand-written custom iterable with for...of, what is logged?

JAVASCRIPT Copy

```
function range(start, end, step = 1) {
  return {
    [Symbol.iterator]() {
      let current = start;
      return {
```

```
            current += step;
            return { value, done: false };
          }
          return { value: undefined, done: true };
        }
      };
    }
  };
}
const out = [];
for (const n of range(0, 10, 3)) out.push(n);
console.log(out);
```

- [ ] `[ 0, 3, 6 ]`
- [ ] `[ 0, 3, 6, 9, 12 ]`
- [ ] `[ 0, 3, 6, 9 ]`
- [ ] `[ 3, 6, 9 ]`

Check answer

‹ Previous
async / await

Next ›
Destructuring, Spread & Rest

LEARN MORE

ARTICLES & DOCS

Generator (MDN)
MDN

VIDEOS



What are generators & iterators?

YouTube



Iterator & generator fundamentals

YouTube



Generators in 7 minutes

YouTube

Hand-picked references for this lesson.